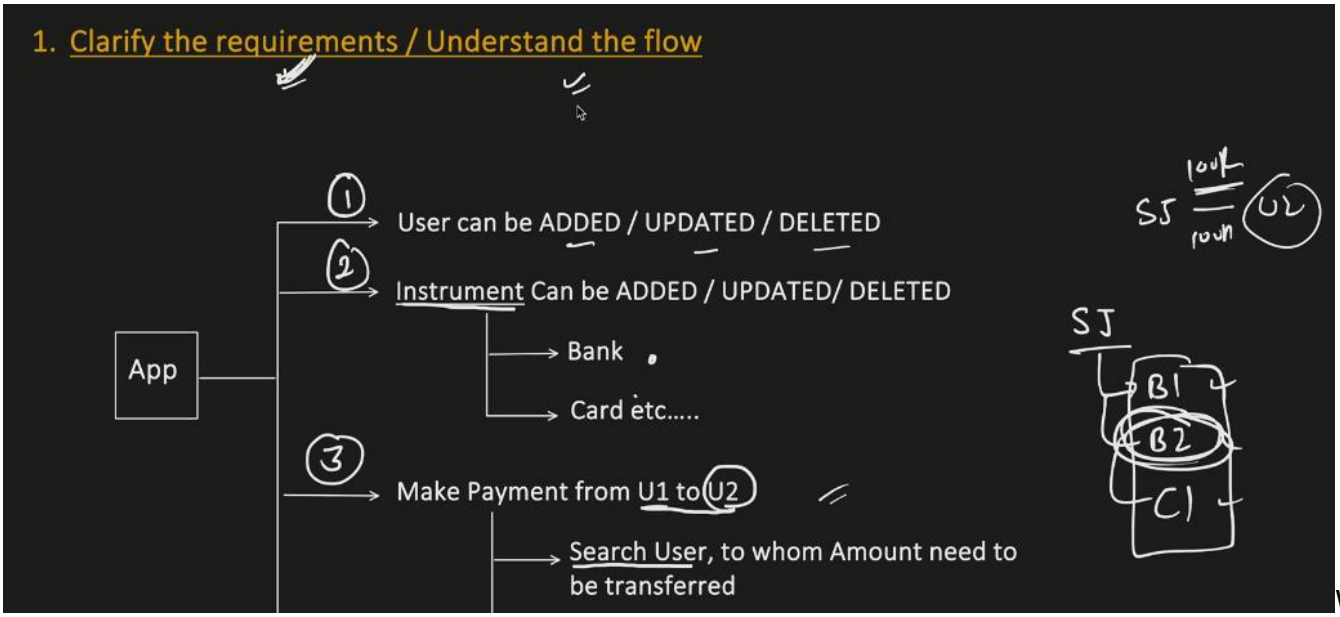
37. LLD of Payment Gateway



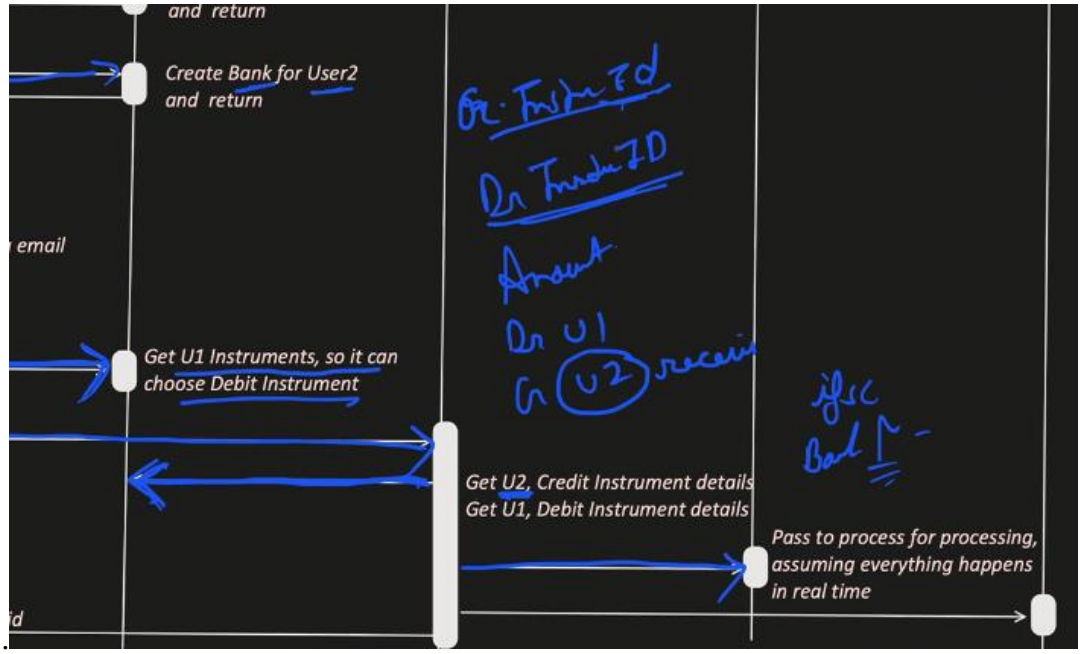
We have created instrument for two users. Suppose user1 wants to make a payment to user2 of ₹10.

It will first search get user details, maybe through email ID or phone number,
So now let's say that it is able to find the user2 details through email ID okay.

It will call Instrument service and getting all the instruments of User1. user1 should be able to fetch the instruments so that it can select from which instrument can be used to fund the transaction.
It will invoke the make payment and it will go to the transaction service.
(Credit to this receiver ID , the amount , debit instrument ID , debit user - sender)
you never select credit instrument . The transaction service will call the Instrument service. What is the default instrument present? (preferred instrument)
If preferred is not set , it will pick one random instrument.

So transaction service definitely needs to call instrument service to fetch the instrument detail because it needs to pass this information to the processor . when transaction service is getting an ID , credit to this instruction ID and debit from this
instruction ID what it will do is it will call this instrument service fetch the details like ifSc and all for the instrument and then process it to a forward it to a processor.

The processor will take it forward to a network and decide whether it is success or failure. We will store the transaction, we will create the transaction, whether it's a success or failure and ultimately we will send the notification.



1. Real-Time (Synchronous) Operations

- **Definition:** Tasks are executed sequentially, one after another, in immediate time.
- **Blocking:** The system waits for the current task to finish before starting the next. If a task takes time, the entire system pauses.

2. Asynchronous Operations

- **Definition:** Tasks run independently of the main program flow and do not need to finish immediately.
- **Non-Blocking:** The system can initiate a task and move on to others without waiting, often using message queues or callbacks to handle the result later.

Feature	Real-Time (Sync)	Asynchronous (Async)
Execution	Sequential (one by one)	Concurrent (simultaneous)
Blocking	Yes (waits for result)	No (continues working)
Responsiveness	Low (can freeze UI)	High (app stays active)
Complexity	Simple	Complex
Ideal For	Immediate feedback/consistency	Long-running/independent tasks

Follow up question can come like:

1. Payment processing can take up to 3-5 days.

   We can make use of ASYC operation. Like

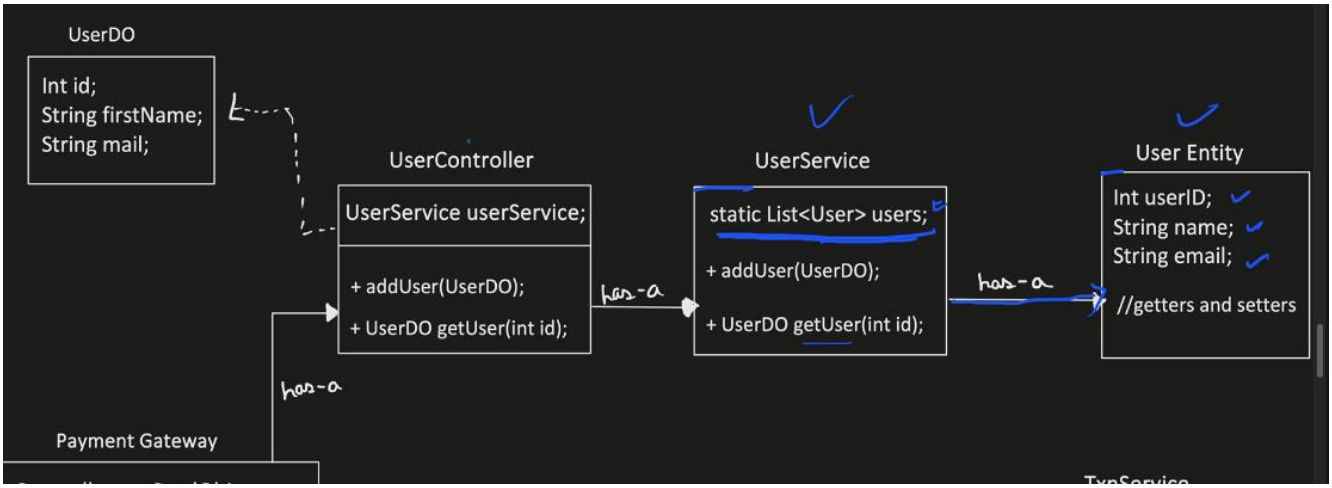
   real time
   i) We will invoke Processor to validate,
      1. if U1 has sufficient money to fund this txn.
      2. If U2 instrument is valid.

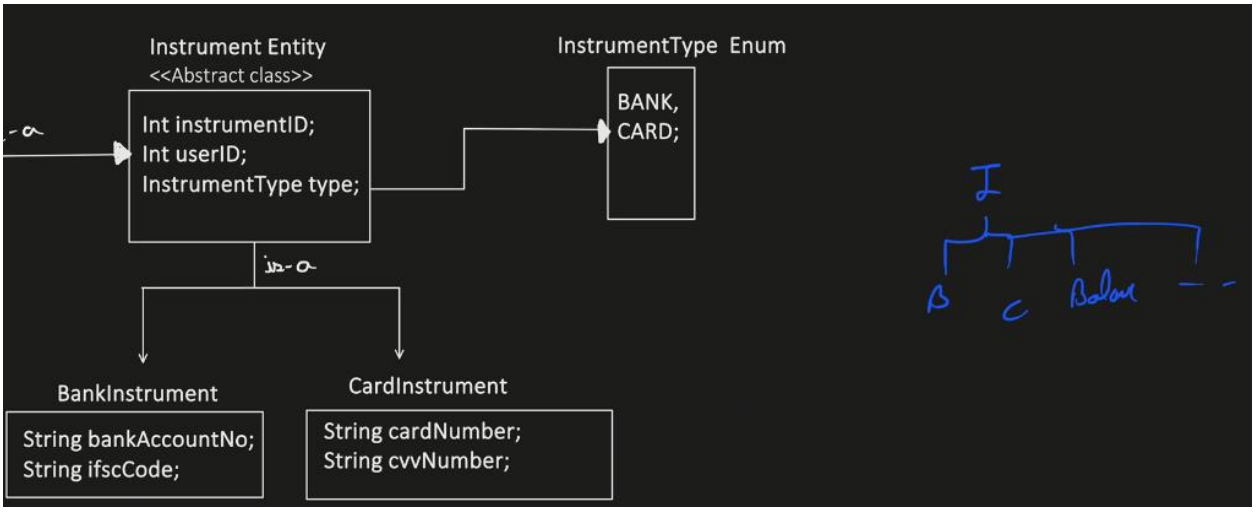
   ii) Then we can save the transaction in PENDING.

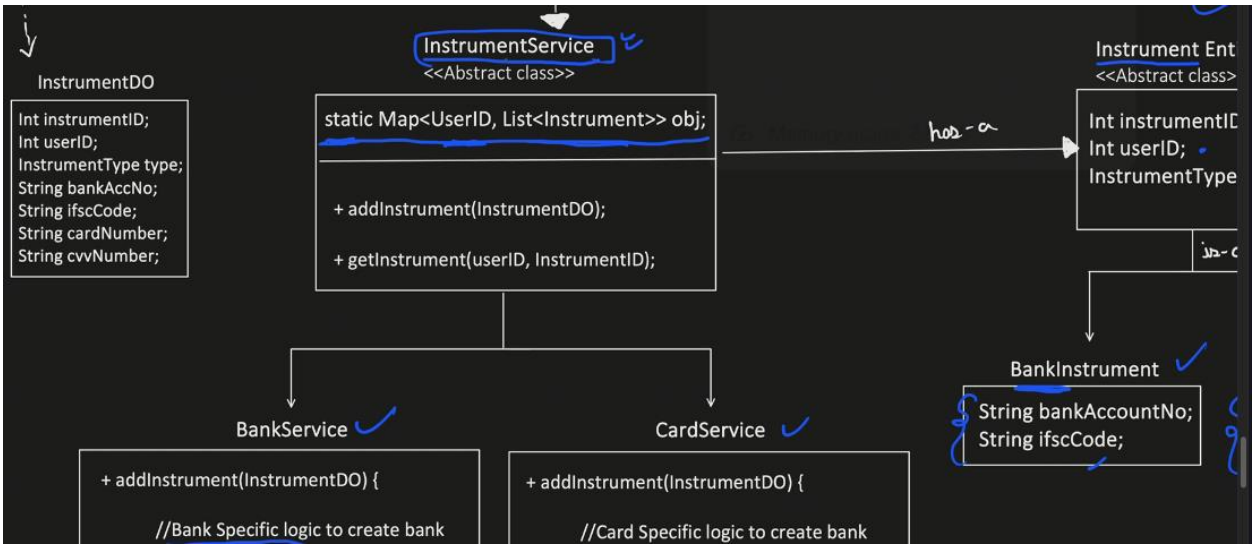
   async
      1. We will invoke the process in ASYNC and complete the txn in PENDING state.
      2. Once Process Invoke our TXN Service and we will update the txn either SUCCESS or FAILURE.

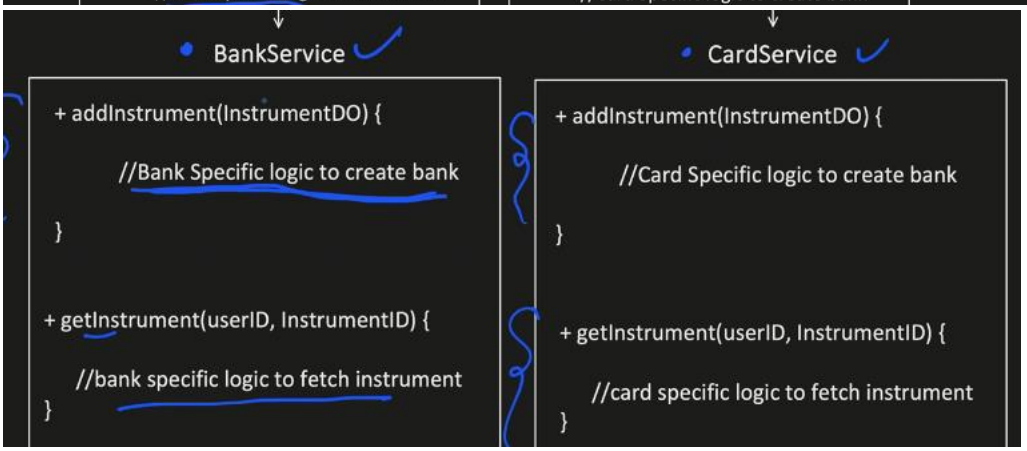
If interviewer put a follow up question, that why we have to wait on a real time if what if it takes time ? you can tell this part async operation that -> we will do validation from processor in real time. And after that, once processor validated, then we will make an async call.

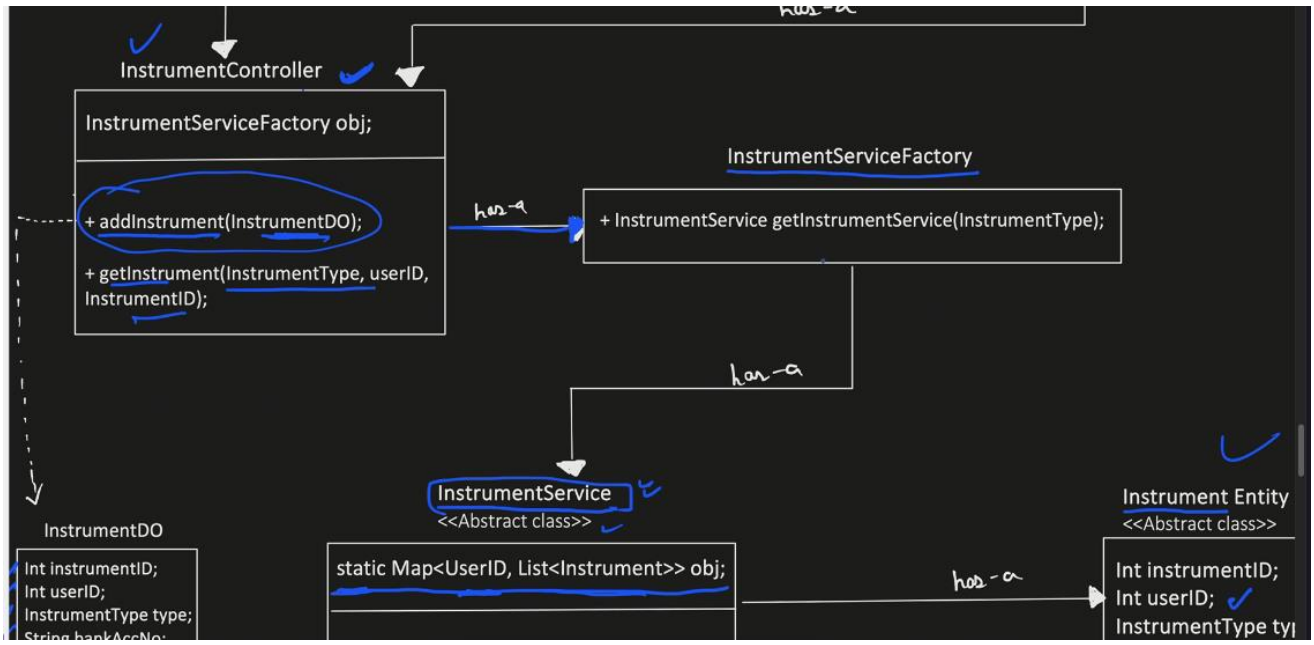
And then we will put the transition into pending and after once the processor processes it and get the response, we will change it to success or failure.

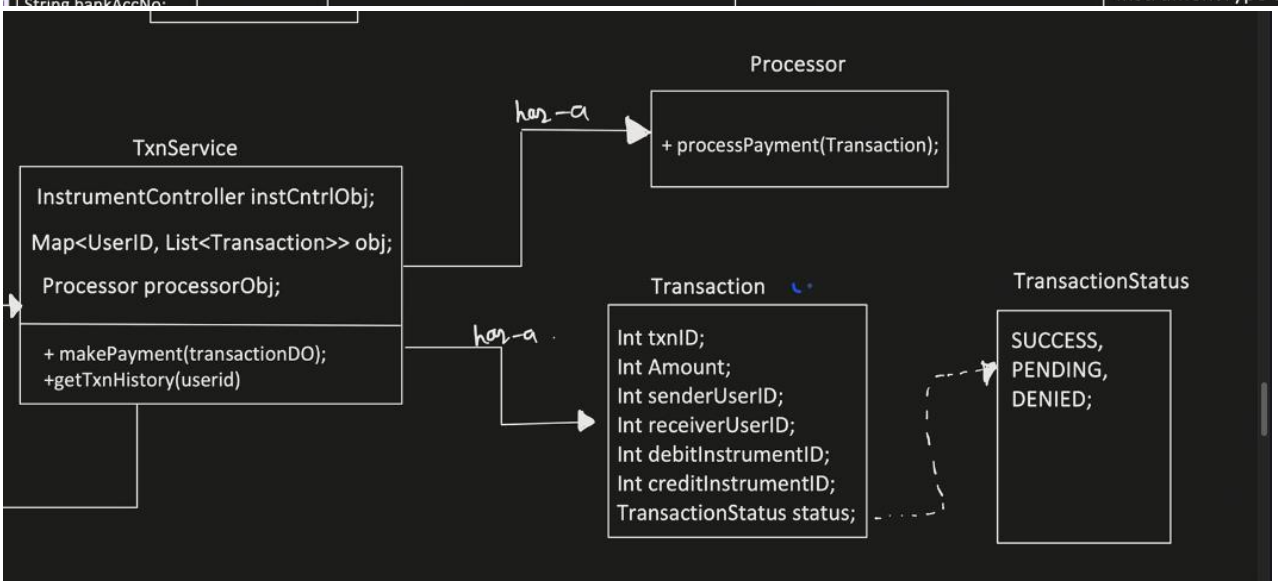


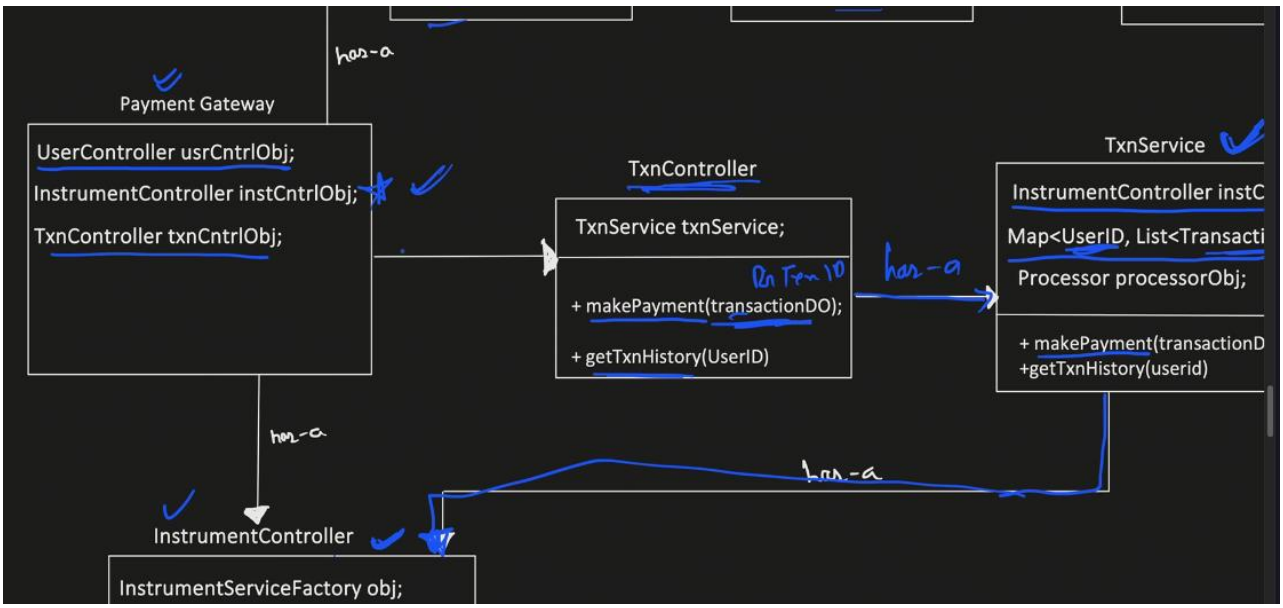

Instrument Entity
<<Abstract class>>
Int instrumentID;
Int userID;
InstrumentType type;
InstrumentType Enum
BANK,
CARD;
is-a
BankInstrument
String bankAccountNo;
String ifscCode;
CardInstrument
String cardNumber;
String cvvNumber;


InstrumentDO
Int instrumentID;
Int userID;
InstrumentType type;
String bankAccNo;
String ifscCode;
String cardNumber;
String cvvNumber;
InstrumentService
<<Abstract class>>
static Map<UserID, List<Instrument>> obj;
+ addInstrument(InstrumentDO);
+ getInstrument(userID, InstrumentID);
has-a
Instrument Ent
<<Abstract class>
Int instrumentID
Int userID;
InstrumentType
BankInstrument
String bankAccountNo;
String ifscCode;
BankService
+ addInstrument(InstrumentDO) {
//Bank Specific logic to create bank
CardService
+ addInstrument(InstrumentDO) {
//Card Specific logic to create bank


BankService
+ addInstrument(InstrumentDO) {
//Bank Specific logic to create bank
}
+ getInstrument(userID, InstrumentID) {
//bank specific logic to fetch instrument
}
CardService
+ addInstrument(InstrumentDO) {
//Card Specific logic to create bank
}
+ getInstrument(userID, InstrumentID) {
//card specific logic to fetch instrument
}


InstrumentController
InstrumentServiceFactory obj;
+ addInstrument(InstrumentDO);
+ getInstrument(InstrumentType, userID, InstrumentID);
has-a
InstrumentServiceFactory
+ InstrumentService getInstrumentService(InstrumentType);
has-a
InstrumentService
<<Abstract class>>
static Map<UserID, List<Instrument>> obj;
InstrumentDO
Int instrumentID;
Int userID;
InstrumentType type;
has-a
Instrument Entity
<<Abstract class>>
Int instrumentID;
Int userID;
InstrumentType ty


Processor
+ processPayment(Transaction);
has-a
TxnService
InstrumentController instCntrlObj;
Map<UserID, List<Transaction>> obj;
Processor processorObj;
+ makePayment(transactionDO);
+getTxnHistory(userid)
has-a
Transaction
Int txnID;
Int Amount;
Int senderUserID;
Int receiverUserID;
Int debitInstrumentID;
Int creditInstrumentID;
TransactionStatus status;
TransactionStatus
SUCCESS,
PENDING,
DENIED;


has-a
Payment Gateway
UserController usrCntrlObj;
InstrumentController instCntrlObj;
TxnController txnCntrlObj;
TxnController
TxnService txnService;
+ makePayment(transactionDO);
+ getTxnHistory(UserID)
has-a
TxnService
InstrumentController instC
Map<UserID, List<Transacti
Processor processorObj;
+ makePayment(transactionD
+getTxnHistory(userid)
has-a
has-a
InstrumentController
InstrumentServiceFactory obj;

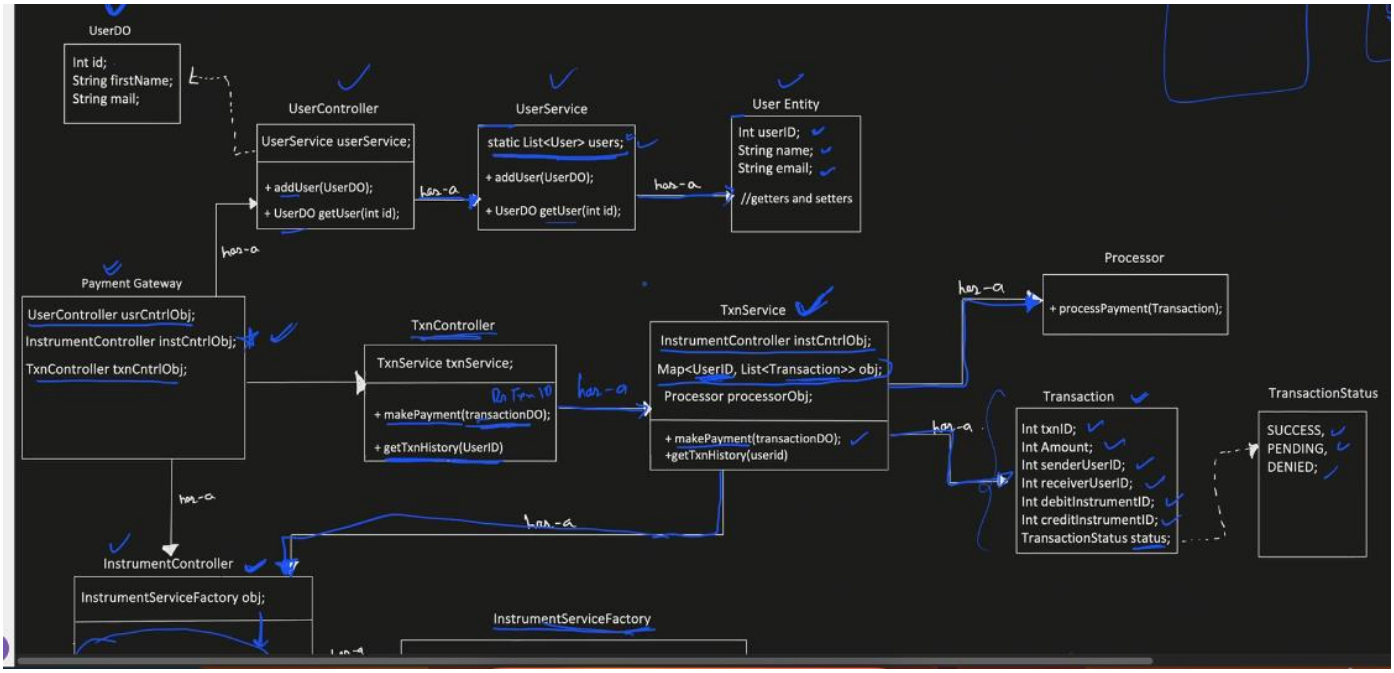
```
learningspringboot ~/PracticeProject/lear
.idea
.mvn
src
main
java
com.conceptandcoding.learni
PaymentSystemLLD
Instrument
Transaction
Processor
Transaction
TransactionController
TransactionDO
TransactionService
TransactionStatus
User
PaymentGateway
resources
test
target
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
External Libraries
Scratches and Consoles
```

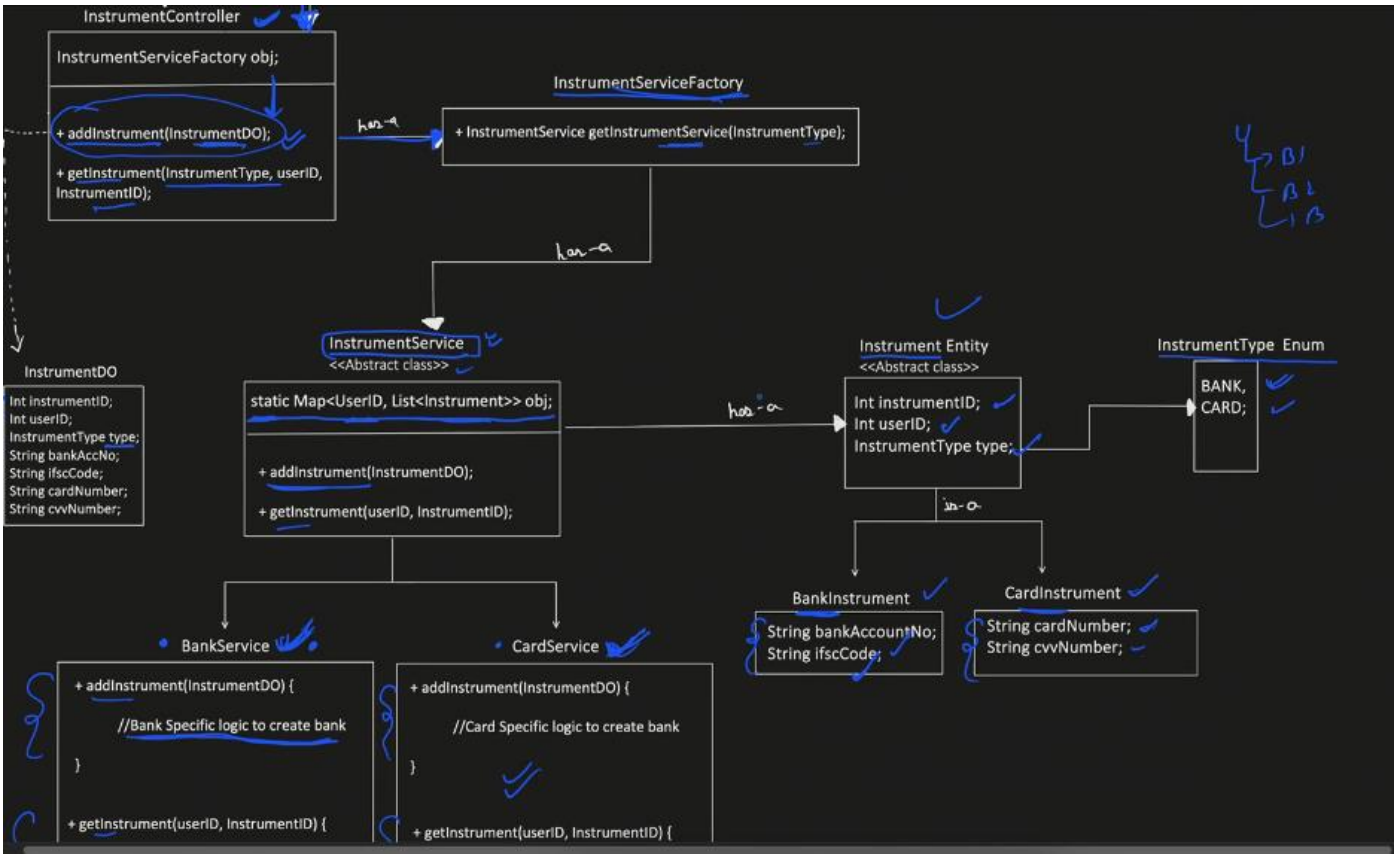
```java
package PaymentSystemLLD.Transaction;

public class TransactionDO {

    int txnID;
    int senderId;
    int receiverId;
    int debitInstrumentId;
    int creditInstrumentId;
    int amount;
    TransactionStatus status;

    public int getTxnID() { return txnID; }

    public void setTxnID(int txnID) { this.txnID = txnID; }

    public int getSenderId() { return senderId; }

    public void setSenderId(int senderId) { this.senderId = senderId; }

    public int getReceiverId() { return receiverId; }

    public void setReceiverId(int receiverId) { this.receiverId = receiverId; }

    public int getDebitInstrumentId() { return debitInstrumentId; }

    public void setDebitInstrumentId(int debitInstrumentId) { this.debitInstrumentId = debitInstrumentId; }

    public int getCreditInstrumentId() { return creditInstrumentId; }

    public void setCreditInstrumentId(int creditInstrumentId) { this.creditInstrumentId = creditInstrumentId; }
```

learningspringboot > src > main > java > PaymentSystemLLD > Transaction > TransactionDO > debitInstrumentId 8:26 (17 chars) LF UTF-

```java
    public TransactionDO(){
        instrumentController = new InstrumentController();
        processor = new Processor();
    }

    public  List<Transaction> getTransactionHistory(int userID) { return userVsTransactionsList.get(userID); }


    public TransactionDO makePayment(TransactionDO txnDO) {

        //validate details

        //load the sender instrument details, which need to be passed to processors
        InstrumentDO senderInstrumentDO = instrumentController.getInstrumentByID(txnDO.getSenderId(), txnDO.getDebitInstrumentId(

        //load the receiver instrument details, which need to be passed to processors
        InstrumentDO receiverInstrumentDO = instrumentController.getInstrumentByID(txnDO.getReceiverId(), txnDO.getCreditInstrume

        //pass the data to processor
        processor.processPayment(senderInstrumentDO, receiverInstrumentDO);

        //based on processor response, we will set the status. for now har coding it to SUCCESS
        Transaction txn = new Transaction();
        txn.setAmount(txnDO.getAmount());
        txn.setTxnID(new Random().nextInt() * (100-10) + 10);
        txn.setSenderId(txnDO.getSenderId());
        txn.setReceiverId(txnDO.getReceiverId());
        txn.setDebitInstrumentId(txnDO.getDebitInstrumentId());
        txn.setCreditInstrumentId(txnDO.getCreditInstrumentId());
        txn.setStatus(TransactionStatus.SUCCESS);

        List<Transaction> senderTxnsList = userVsTransactionsList.get(txn.getSenderId());
```

```
Bank Instrument created for User1: 73
Card Instrument created for User2: 54
User1 InstID: 73: UserID: 84: InstrumentType: BANK
User2 InstID: 54: UserID: 17: InstrumentType: CARD
User1 txnID: 63: Amount: 10: receiver: 17

Process finished with exit code 0
```





Zoho-notebook :

https://notebook.zohopublic.in/public/notes/bietv6993f5a023ec4e9f89436149ac3ed2f7

38. Object Pool Design Pattern (Creational Design Pattern)

https://notebook.zohopublic.in/public/notes/dcr5za494c035adb441818e2bc20485b4de58

https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/creationalpatterns/objectpool?ref_type=heads

1. Thread (very basic)

A **thread** is a small unit of work that runs inside a program.

- One program can run **many threads at the same time**
- Each thread executes code independently

Simple example

- Thread 1 → downloading data
- Thread 2 → updating UI
- Thread 3 → saving file

All run "in parallel" (concurrently)

---

2. Why threads cause problems

When threads **share data**, they can interfere.

Example

```
count = 0

Thread A → read 0 → +1 → write 1
Thread B → read 0 → +1 → write 1
```

Expected: `2`

Actual: `1` ❌

This is called a **race condition**

→ Multiple threads "race" and overwrite each other

---

3. synchronized (control mechanism)

`synchronized` ensures:

👉 Only **one thread at a time** executes a block of code

Example

```java
public synchronized void increment() {
    count++;
}
```

Now:

- Thread A runs fully
- Then Thread B runs

✔ No data corruption

✔ Safe execution

Simple analogy

- Without synchronized → 5 people writing on same whiteboard at once
- With synchronized → only 1 person allowed at a time

---

4. Thread-safe

A code is **thread-safe** if:

👉 It works correctly even when multiple threads use it

Using `synchronized` makes code thread-safe

---

5. Lazy Initialization

Lazy = **create only when needed**

Example

```java
if (object == null) {
    object = new Object();
}
```

- Object is NOT created at start
- Created only when first requested

---

6. Problem in Lazy Initialization (multi-threading)

Two threads come at same time:

```java
if (instance == null)
```

Both see `null`

Both create object ❌

👉 Now you have **2 objects instead of 1**

---

7. Double-Checked Locking (solution)

```java
if (instance == null) {                // Check 1
    synchronized (ClassName.class) {
        if (instance == null) {        // Check 2
            instance = new ClassName();
        }
    }
}
```

Why 2 checks?

First check (outside)

- Avoids locking every time
- Improves performance

Second check (inside)

- Ensures only one thread creates object

Flow

1. Thread A → sees `null`
2. Thread B → also sees `null`
3. Thread A enters synchronized → creates object
4. Thread B enters later → sees NOT null → does nothing

✔ Only one object created

✔ Efficient + safe

---

8. Thread-safe Lazy Initialization (final concept)

👉 Combine:

- Lazy initialization (create when needed)
- Thread safety (using synchronized)

✔ Result:

- Object created only once
- No duplicate creation
- Safe in multi-threading

Final One-line Summary

- **Thread** → independent worker
- **synchronized** → only one worker allowed at a time
- **Lazy initialization** → create when needed
- **Double-checked locking** → efficient + safe way to do lazy initialization

Code :

```java
private static DBConnectionPoolManager dbConnectionPoolManagerInstance = null;
```

- Stores the **single instance** (Singleton)

3. getInstance() → Singleton

```java
if(instance == null) {
    synchronized(...) {
        if(instance == null) {
            instance = new DBConnectionPoolManager();
        }
    }
}
```

👉 Purpose:

- Only **one pool manager** should exist

👉 Flow:

1. First call → instance is null → create object
2. Later calls → return same object

👉 Uses:

- **Double-checked locking**
- **Thread-safe lazy initialization**

1. Thread

- Multiple users/requests can call this at same time

2. synchronized

- Ensures:
  👉 Only one thread modifies pool at a time
- Prevents errors like:
  - Giving same connection to 2 users

3. Singleton

- Only one pool manager exists

4. Double-Checked Locking

- Used in `getInstance()`
- Ensures:
  - Safe creation
  - Better performance (no locking every time)

5. Lazy Initialization

- Pool is created **only when first needed**

4. getDBConnection() → get a connection

```java
public synchronized DBConnection getDBConnection()
```

👉 `synchronized`:

- Only **one thread at a time** can execute this method

5. releaseDBConnection() → return connection

```java
public synchronized void releaseDBConnection(DBConnection dbConnection)
```

👉 Action:

1. Remove from **in-use list**
2. Add back to **free pool**

What happens if `getInstance()` is called 3 times?

Short answer

👉 All 3 calls return **the same single object**
👉 The constructor runs **only once**

Step-by-step (simple)

First call

```java
DBConnectionPoolManager.getInstance();
```

- `instance == null` ✅
- Enters `synchronized` block
- Creates new object
- Stores it in `instance`

✔ Pool initialized (3 connections created)

Second call

```java
DBConnectionPoolManager.getInstance();
```

- `instance != null` ❌
- Skips creation
- Directly returns existing object

Third call

Same as second call:

- No new object created
- Returns same instance

Third call

Same as second call:

- No new object created
- Returns same instance

Visual idea

```
Call 1 → creates object  (NEW)
Call 2 → returns same   (OLD)
Call 3 → returns same   (OLD)
```

Important point

Even if **multiple threads call at the same time**:

- Only **one object** will be created
- Because of:
  - `synchronized`
  - double-checked locking

Final conclusion

- Number of calls = 3
- Number of objects created = **1**
- All calls share the **same connection pool**

One-line understanding

👉 `getInstance()` always gives the **same object**, no matter how many times you call it.